Algoritmos de optimización - Trabajo práctico

Nombre: Miguel Laversa
Problema: 2. Programación de partidos de La Liga

El objetivo es asignar los diez partidos de una jornada a los horarios disponibles de forma que se maximice la audiencia total. Los cálculos se expresan en millones de espectadores.

La tabla de audiencias del enunciado queda cortada antes de mostrar el cruce C-C. El valor de 0,47 millones se obtiene de la tabla general del mismo documento: Mallorca-Eibar y Leganés-Osasuna aparecen como partidos C-C con esa audiencia base.

```
from collections import Counter
from itertools import product
from math import comb
from random import Random
from time import perf_counter

import pandas as pd

HORARIOS = (
    ("V20", "Viernes 20:00", 0.40),
    ("S12", "Sábado 12:00", 0.55),
    ("S16", "Sábado 16:00", 0.70),
    ("S18", "Sábado 18:00", 0.80),
    ("S20", "Sábado 20:00", 1.00),
    ("D12", "Domingo 12:00", 0.45),
    ("D16", "Domingo 16:00", 0.75),
    ("D18", "Domingo 18:00", 0.85),
    ("D20", "Domingo 20:00", 1.00),
    ("L20", "Lunes 20:00", 0.40),
)

AUDIENCIA_BASE = {
    ("A", "A"): 2.00,
    ("A", "B"): 1.30,
    ("A", "C"): 1.00,
    ("B", "B"): 0.90,
    ("B", "C"): 0.75,
    ("C", "C"): 0.47,
}

PENALIZACION = {
    0: 0.00,
    1: 0.25,
    2: 0.45,
    3: 0.60,
    4: 0.70,
    5: 0.75,
    6: 0.78,
```

```
    7: 0.80,
    8: 0.80,
}

PARTIDOS_EJEMPLO = (
    ("Celta - Real Madrid", "B", "A"),
    ("Valencia - R. Sociedad", "B", "A"),
    ("Mallorca - Eibar", "C", "C"),
    ("Athletic - Barcelona", "B", "A"),
    ("Leganés - Osasuna", "C", "C"),
    ("Villarreal - Granada", "B", "C"),
    ("Alavés - Levante", "B", "B"),
    ("Espanyol - Sevilla", "B", "B"),
    ("Betis - Valladolid", "B", "C"),
    ("Atlético - Getafe", "B", "B"),
)


def audiencia_base(categoria_local, categoria_visitante):
    categorias = tuple(sorted((categoria_local, categoria_visitante)))
    return AUDIENCIA_BASE[categorias]


def preparar_partidos(partidos):
    return tuple(
        {
            "partido": nombre,
            "categorias": f"{local}-{visitante}",
            "base": audiencia_base(local, visitante),
        }
        for nombre, local, visitante in partidos
    )


partidos_ejemplo = preparar_partidos(PARTIDOS_EJEMPLO)

display(pd.DataFrame(HORARIOS, columns=["Código", "Horario", "Ponderación"]))
display(pd.DataFrame(partidos_ejemplo))
```

	Código	Horario	Ponderación
0	V20	Viernes 20:00	0.40
1	S12	Sábado 12:00	0.55
2	S16	Sábado 16:00	0.70
3	S18	Sábado 18:00	0.80
4	S20	Sábado 20:00	1.00
5	D12	Domingo 12:00	0.45
6	D16	Domingo 16:00	0.75
7	D18	Domingo 18:00	0.85
8	D20	Domingo 20:00	1.00
9	L20	Lunes 20:00	0.40

	partido	categorias	base
0	Celta - Real Madrid	B-A	1.30
1	Valencia - R. Sociedad	B-A	1.30
2	Mallorca - Eibar	C-C	0.47
3	Athletic - Barcelona	B-A	1.30
4	Leganés - Osasuna	C-C	0.47
5	Villarreal - Granada	B-C	0.75
6	Alavés - Levante	B-B	0.90
7	Espanyol - Sevilla	B-B	0.90
8	Betis - Valladolid	B-C	0.75
9	Atlético - Getafe	B-B	0.90

1. Número de posibilidades

(*) ¿Cuántas posibilidades hay sin tener en cuenta las restricciones? ¿Cuántas hay teniendo en cuenta todas las restricciones?

Cada uno de los diez partidos puede asignarse a cualquiera de los diez horarios. Sin restricciones existen **10^{10} = 10.000.000.000** posibilidades.

Interpreto la indicación «un partido el viernes y un partido el lunes» como **exactamente uno en cada día**, de acuerdo con la jornada de ejemplo. Se elige primero el partido del viernes, después el del lunes y cada uno de los ocho restantes puede ocupar cualquiera de los ocho horarios del fin de semana:

$10 \cdot 9 \cdot 8^8 = 1.509.949.440$ posibilidades

La coincidencia de partidos no invalida una solución. Únicamente reduce su audiencia.

```
n_partidos = 10
n_horarios = len(HORARIOS)

posibilidades_sin_restricciones = n_horarios ** n_partidos
posibilidades_factibles = (
    n_partidos
    * (n_partidos - 1)
    * (n_horarios - 2) ** (n_partidos - 2)
)

pd.DataFrame(
    {
        "Espacio": [
            "Sin restricciones",
            "Con un partido el viernes y otro el lunes",
        ],
        "Posibilidades": [
            posibilidades_sin_restricciones,
            posibilidades_factibles,
        ],
    }
).style.format({"Posibilidades": "{:,.0f}"})
```

	Espacio	Posibilidades
0	Sin restricciones	10,000,000,000
1	Con un partido el viernes y otro el lunes	1,509,949,440

2. Modelo para el espacio de soluciones

(*) ¿Cuál es la estructura de datos que mejor se adapta al problema?

Represento los partidos mediante una secuencia de registros con su nombre, categorías y audiencia base. Los horarios se almacenan en una tupla con código, descripción y ponderación. Una solución es un vector de longitud (n): la posición (i) identifica un partido y el valor almacenado es el índice de su horario.

Esta representación ocupa $O(n + m)$ y permite contar las coincidencias con un diccionario en $O(n)$. Una matriz binaria de $n \times m$ también sería válida, pero introduciría información redundante porque cada partido solo puede tener un horario.

```
def audiencia_total(partidos, solucion):
    ocupacion = Counter(solucion)
    total = 0.0

    for partido, indice_horario in zip(partidos, solucion):
```

```
        coincidencias = ocupacion[indice_horario] - 1
        ponderacion = HORARIOS[indice_horario][2]
        reduccion = PENALIZACION[coincidencias]
        total += partido["base"] * ponderacion * (1 - reduccion)

    return total


def tabla_solucion(partidos, solucion):
    ocupacion = Counter(solucion)
    filas = []

    for partido, indice_horario in zip(partidos, solucion):
        codigo, horario, ponderacion = HORARIOS[indice_horario]
        coincidencias = ocupacion[indice_horario] - 1
        factor = ponderacion * (1 - PENALIZACION[coincidencias])
        filas.append(
            {
                "Partido": partido["partido"],
                "Categorías": partido["categorias"],
                "Horario": horario,
                "Base": partido["base"],
                "Coincidencias": coincidencias,
                "Factor efectivo": factor,
                "Audiencia": partido["base"] * factor,
            }
        )

    return (
        pd.DataFrame(filas)
        .sort_values(["Horario", "Audiencia"], ascending=[True, False])
        .reset_index(drop=True)
    )
```

3. Función objetivo

(*) ¿Cuál es la función objetivo? ¿Es un problema de maximización o minimización?

Sea $\mathbf{x_i}$ el horario asignado al partido **i**, $\mathbf{b_i}$ su audiencia base, $\mathbf{w_s}$ la ponderación del horario **s**, $\mathbf{k_s}$ el número de partidos asignados a ese horario y **p(c)** la penalización por **c** coincidencias. La función objetivo es:

$$\max A(x) = \Sigma_i \, b_i \cdot w_{(x_i)} \cdot (1 - p(k_{(x_i)} - 1))$$

Las restricciones exigen asignar un único horario a cada partido y colocar exactamente un partido el viernes y otro el lunes. Es un problema de **maximización** de audiencia.

```
solucion_del_enunciado = (0, 1, 2, 3, 4, 6, 6, 7, 8, 9)
audiencia_enunciado = audiencia_total(partidos_ejemplo, solucion_del_enunciado)
```

```
print(f"Audiencia de la jornada de ejemplo: {audiencia_enunciado:.3f} millones")
display(tabla_solucion(partidos_ejemplo, solucion_del_enunciado))
```

```
Audiencia de la jornada de ejemplo: 5.877 millones
```

	Partido	Categorías	Horario	Base	Coincidencias	Factor efectivo	Audiencia
0	Alavés - Levante	B-B	Domingo 16:00	0.90	1	0.5625	0.506250
1	Villarreal - Granada	B-C	Domingo 16:00	0.75	1	0.5625	0.421875
2	Espanyol - Sevilla	B-B	Domingo 18:00	0.90	0	0.8500	0.765000
3	Betis - Valladolid	B-C	Domingo 20:00	0.75	0	1.0000	0.750000
4	Atlético - Getafe	B-B	Lunes 20:00	0.90	0	0.4000	0.360000
5	Valencia - R. Sociedad	B-A	Sábado 12:00	1.30	0	0.5500	0.715000
6	Mallorca - Eibar	C-C	Sábado 16:00	0.47	0	0.7000	0.329000
7	Athletic - Barcelona	B-A	Sábado 18:00	1.30	0	0.8000	1.040000
8	Leganés - Osasuna	C-C	Sábado 20:00	0.47	0	1.0000	0.470000
9	Celta - Real Madrid	B-A	Viernes 20:00	1.30	0	0.4000	0.520000

4. Algoritmo por fuerza bruta

Diseña un algoritmo para resolver el problema por fuerza bruta

La fuerza bruta recorre todas las soluciones factibles. Para cada pareja ordenada de partidos fija uno el viernes y otro el lunes. Después genera todas las asignaciones posibles de los partidos restantes a los ocho horarios del fin de semana. Cada candidato se evalúa por completo y se conserva el de mayor audiencia.

La instancia completa requiere más de 1.500 millones de evaluaciones. Por este motivo ejecuto el algoritmo sobre seis partidos, un tamaño suficiente para comprobar posteriormente que el método mejorado mantiene la solución óptima.

```
INDICE_VIERNES = 0
INDICE_LUNES = 9
HORARIOS_FIN_DE_SEMANA = tuple(range(1, 9))
```

```python
def fuerza_bruta(partidos):
    n = len(partidos)
    mejor_valor = float("-inf")
    mejor_solucion = None
    evaluadas = 0

    for partido_viernes in range(n):
        for partido_lunes in range(n):
            if partido_lunes == partido_viernes:
                continue

            restantes = [
                i
                for i in range(n)
                if i not in (partido_viernes, partido_lunes)
            ]

            for asignacion in product(
                HORARIOS_FIN_DE_SEMANA,
                repeat=n - 2,
            ):
                solucion = [None] * n
                solucion[partido_viernes] = INDICE_VIERNES
                solucion[partido_lunes] = INDICE_LUNES

                for indice_partido, indice_horario in zip(restantes, asignacion):
                    solucion[indice_partido] = indice_horario

                valor = audiencia_total(partidos, solucion)
                evaluadas += 1

                if valor > mejor_valor:
                    mejor_valor = valor
                    mejor_solucion = tuple(solucion)

    return mejor_solucion, mejor_valor, evaluadas


partidos_reducidos = partidos_ejemplo[:6]
inicio = perf_counter()
solucion_bruta, valor_bruto, evaluadas_bruta = fuerza_bruta(partidos_reducidos)
tiempo_bruta = perf_counter() - inicio

print(f"Soluciones evaluadas: {evaluadas_bruta:,}")
print(f"Mejor audiencia: {valor_bruto:.3f} millones")
print(f"Tiempo: {tiempo_bruta:.3f} segundos")
display(tabla_solucion(partidos_reducidos, solucion_bruta))
```

```
Soluciones evaluadas: 122,880
Mejor audiencia: 4.681 millones
Tiempo: 0.256 segundos
```

	Partido	Categorías	Horario	Base	Coincidencias	Factor efectivo	Audiencia
0	Valencia - R. Sociedad	B-A	Domingo 18:00	1.30	0	0.85	1.105
1	Athletic - Barcelona	B-A	Domingo 20:00	1.30	0	1.00	1.300
2	Leganés - Osasuna	C-C	Lunes 20:00	0.47	0	0.40	0.188
3	Villarreal - Granada	B-C	Sábado 18:00	0.75	0	0.80	0.600
4	Celta - Real Madrid	B-A	Sábado 20:00	1.30	0	1.00	1.300
5	Mallorca - Eibar	C-C	Viernes 20:00	0.47	0	0.40	0.188

5. Complejidad del algoritmo por fuerza bruta

Calcula la complejidad del algoritmo por fuerza bruta

Con **n** partidos y **m** horarios, la generación restringida produce:

$$\mathbf{S(n,m) = n(n - 1)(m - 2)^{n-2}}$$

soluciones. Evaluar cada una cuesta **O(n)**, por lo que el tiempo es:

$$\mathbf{O(n^2(n - 1)(m - 2)^{n-2})}$$

El crecimiento es exponencial respecto a **n**. El algoritmo conserva una solución y un contador de ocupación, de modo que el espacio auxiliar es **O(n + m)**.

```
pd.DataFrame(
    {
        "Partidos": range(4, 11),
        "Soluciones factibles": [
            n * (n - 1) * 8 ** (n - 2)
            for n in range(4, 11)
        ],
    }
).style.format({"Soluciones factibles": "{:,.0f}"})
```

	Partidos	Soluciones factibles
0	4	768
1	5	10,240
2	6	122,880
3	7	1,376,256
4	8	14,680,064
5	9	150,994,944
6	10	1,509,949,440

6. Algoritmo mejorado

(*) Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta

La audiencia de un partido depende de su audiencia base y del factor efectivo del horario. Para un patrón de ocupación conocido, todos los partidos que comparten horario reciben el mismo factor:

$f_s(k_s) = w_s \cdot (1 - p(k_s - 1))$

No es necesario probar todas las asignaciones etiquetadas. Primero enumero las distribuciones de los **n - 2** partidos restantes entre los **m - 2** horarios del fin de semana. Para cada distribución construyo una lista con los factores efectivos disponibles y asigno las mayores audiencias base a los mayores factores.

Esta asignación es óptima para el patrón fijado. Si dos audiencias **a ≥ b** estuvieran asociadas a factores **u < v**, intercambiarlas cambiaría el resultado en **(a - b)(v - u) ≥ 0**. Por tanto, eliminar sucesivamente estas inversiones conduce al emparejamiento ordenado sin reducir la audiencia.

El procedimiento sigue siendo exacto: revisa todos los patrones de ocupación y obtiene la mejor asignación dentro de cada uno.

```
def composiciones(total, partes, prefijo=()):
    if partes == 1:
        yield prefijo + (total,)
        return

    for valor in range(total + 1):
        yield from composiciones(
            total - valor,
            partes - 1,
            prefijo + (valor,),
        )


def optimizar_por_ocupacion(partidos):
```

```
    n = len(partidos)
    partidos_ordenados = sorted(
        enumerate(partidos),
        key=lambda elemento: elemento[1]["base"],
        reverse=True,
    )

    mejor_valor = float("-inf")
    mejor_solucion = None
    mejor_ocupacion = None
    patrones_evaluados = 0

    for ocupacion_fin_semana in composiciones(n - 2, len(HORARIOS) - 2):
        ocupacion = (1,) + ocupacion_fin_semana + (1,)
        factores = []

        for indice_horario, cantidad in enumerate(ocupacion):
            if cantidad == 0:
                continue

            coincidencias = cantidad - 1
            ponderacion = HORARIOS[indice_horario][2]
            factor = ponderacion * (1 - PENALIZACION[coincidencias])
            factores.extend((factor, indice_horario) for _ in range(cantidad))

        factores.sort(reverse=True)
        valor = sum(
            partido["base"] * factor
            for (_, partido), (factor, _) in zip(partidos_ordenados, factores)
        )
        patrones_evaluados += 1

        if valor > mejor_valor:
            solucion = [None] * n
            for (indice_partido, _), (_, indice_horario) in zip(
                partidos_ordenados,
                factores,
            ):
                solucion[indice_partido] = indice_horario

            mejor_valor = valor
            mejor_solucion = tuple(solucion)
            mejor_ocupacion = ocupacion

    return (
        mejor_solucion,
        mejor_valor,
        mejor_ocupacion,
        patrones_evaluados,
    )


inicio = perf_counter()
(
    solucion_mejorada,
    valor_mejorado,
```

```
    ocupacion_mejorada,
    patrones_mejorados,
) = optimizar_por_ocupacion(partidos_ejemplo)
tiempo_mejorado = perf_counter() - inicio

print(f"Patrones evaluados: {patrones_mejorados:,}")
print(f"Mejor audiencia: {valor_mejorado:.3f} millones")
print(f"Tiempo: {tiempo_mejorado:.4f} segundos")
display(tabla_solucion(partidos_ejemplo, solucion_mejorada))
```

```
Patrones evaluados: 6,435
Mejor audiencia: 6.856 millones
Tiempo: 0.0311 segundos
```

	Partido	Categorías	Horario	Base	Coincidencias	Factor efectivo	Audiencia
0	Betis - Valladolid	B-C	Domingo 12:00	0.75	0	0.45	0.3375
1	Espanyol - Sevilla	B-B	Domingo 16:00	0.90	0	0.75	0.6750
2	Athletic - Barcelona	B-A	Domingo 18:00	1.30	0	0.85	1.1050
3	Celta - Real Madrid	B-A	Domingo 20:00	1.30	0	1.00	1.3000
4	Mallorca - Eibar	C-C	Lunes 20:00	0.47	0	0.40	0.1880
5	Villarreal - Granada	B-C	Sábado 12:00	0.75	0	0.55	0.4125
6	Atlético - Getafe	B-B	Sábado 16:00	0.90	0	0.70	0.6300
7	Alavés - Levante	B-B	Sábado 18:00	0.90	0	0.80	0.7200
8	Valencia - R. Sociedad	B-A	Sábado 20:00	1.30	0	1.00	1.3000
9	Leganés - Osasuna	C-C	Viernes 20:00	0.47	0	0.40	0.1880

7. Complejidad del algoritmo mejorado

(*) Calcula la complejidad del algoritmo

Distribuir **n - 2** partidos indistinguibles entre **m - 2** horarios equivale a una composición débil. El número de patrones es:

P(n,m) = C(n + m - 5, m - 3)

Para cada patrón se construyen y ordenan **n** factores, por lo que el tiempo es:

O(C(n + m - 5, m - 3) · n log n)

El espacio auxiliar es **O(n + m)**. Para **n = m = 10** se revisan **C(15, 7) = 6.435** patrones, frente a 1.509.949.440 asignaciones de la fuerza bruta. Con un número fijo de horarios, la cantidad de patrones crece polinómicamente respecto al número de partidos.

```
filas_validacion = []

for n in range(3, 7):
    caso = partidos_ejemplo[:n]
    _, valor_fuerza_bruta, evaluaciones = fuerza_bruta(caso)
    (
        solucion_exacta,
        valor_exacto,
        _,
        patrones,
    ) = optimizar_por_ocupacion(caso)

    coinciden = abs(valor_fuerza_bruta - valor_exacto) < 1e-12
    assert coinciden
    assert Counter(solucion_exacta)[INDICE_VIERNES] == 1
    assert Counter(solucion_exacta)[INDICE_LUNES] == 1

    filas_validacion.append(
        {
            "Partidos": n,
            "Evaluaciones de fuerza bruta": evaluaciones,
            "Patrones de ocupación": patrones,
            "Óptimo de fuerza bruta": valor_fuerza_bruta,
            "Óptimo del algoritmo exacto": valor_exacto,
            "Coinciden": coinciden,
        }
    )

validacion = pd.DataFrame(filas_validacion)
display(
    validacion.style.format(
        {
            "Evaluaciones de fuerza bruta": "{:,.0f}",
            "Patrones de ocupación": "{:,.0f}",
            "Óptimo de fuerza bruta": "{:.3f}",
            "Óptimo del algoritmo exacto": "{:.3f}",
        }
    )
)
print("El algoritmo exacto reproduce el óptimo de fuerza bruta en los cuatro casos.")
```

	Partidos	Evaluaciones de fuerza bruta	Patrones de ocupación	Óptimo de fuerza bruta	Óptimo del algoritmo exacto	Coinciden
0	3	48	8	2.008	2.008	True
1	4	768	36	3.308	3.308	True
2	5	10,240	120	4.081	4.081	True
3	6	122,880	330	4.681	4.681	True

`El algoritmo exacto reproduce el óptimo de fuerza bruta en los cuatro casos.`

8. Juego de datos aleatorio

Según el problema, diseña un juego de datos de entrada aleatorios

Genero una jornada con veinte equipos: tres de categoría A, once de categoría B y seis de categoría C, como indica el enunciado. Una semilla fija permite reproducir el experimento. Los equipos se mezclan y se emparejan de dos en dos. La audiencia base se obtiene de la tabla de categorías.

```
def generar_jornada(semilla=42):
    equipos = (
        [(f"Equipo A{i}", "A") for i in range(1, 4)]
        + [(f"Equipo B{i}", "B") for i in range(1, 12)]
        + [(f"Equipo C{i}", "C") for i in range(1, 7)]
    )

    generador = Random(semilla)
    generador.shuffle(equipos)

    partidos = []
    for i in range(0, len(equipos), 2):
        local, visitante = equipos[i], equipos[i + 1]
        partidos.append(
            (
                f"{local[0]} - {visitante[0]}",
                local[1],
                visitante[1],
            )
        )

    return preparar_partidos(partidos)


partidos_aleatorios = generar_jornada()
display(pd.DataFrame(partidos_aleatorios))
```

	partido	categorias	base
0	Equipo C6 - Equipo B3	C-B	0.75
1	Equipo C1 - Equipo B2	C-B	0.75
2	Equipo B7 - Equipo B11	B-B	0.90
3	Equipo C2 - Equipo C5	C-C	0.47
4	Equipo B4 - Equipo B10	B-B	0.90
5	Equipo C4 - Equipo B8	C-B	0.75
6	Equipo A2 - Equipo B9	A-B	1.30
7	Equipo A3 - Equipo C3	A-C	1.00
8	Equipo B5 - Equipo B6	B-B	0.90
9	Equipo A1 - Equipo B1	A-B	1.30

9. Aplicación al juego de datos generado

Aplica el algoritmo al juego de datos generado

Aplico el algoritmo exacto basado en patrones de ocupación. Además de la audiencia máxima, compruebo que todos los partidos aparecen una sola vez y que se cumplen las asignaciones obligatorias del viernes y del lunes.

```
(
    solucion_aleatoria,
    valor_aleatorio,
    ocupacion_aleatoria,
    patrones_aleatorios,
) = optimizar_por_ocupacion(partidos_aleatorios)

conteo_aleatorio = Counter(solucion_aleatoria)
assert len(solucion_aleatoria) == len(partidos_aleatorios)
assert conteo_aleatorio[INDICE_VIERNES] == 1
assert conteo_aleatorio[INDICE_LUNES] == 1

print(f"Patrones evaluados: {patrones_aleatorios:,}")
print(f"Audiencia máxima: {valor_aleatorio:.3f} millones")
display(tabla_solucion(partidos_aleatorios, solucion_aleatoria))
```

```
Patrones evaluados: 6,435
Audiencia máxima: 6.713 millones
```

	Partido	Categorías	Horario	Base	Coincidencias	Factor efectivo	Audiencia
0	Equipo C1 - Equipo B2	C-B	Domingo 12:00	0.75	0	0.45	0.3375
1	Equipo B4 - Equipo B10	B-B	Domingo 16:00	0.90	0	0.75	0.6750
2	Equipo A3 - Equipo C3	A-C	Domingo 18:00	1.00	0	0.85	0.8500
3	Equipo A2 - Equipo B9	A-B	Domingo 20:00	1.30	0	1.00	1.3000
4	Equipo C4 - Equipo B8	C-B	Lunes 20:00	0.75	0	0.40	0.3000
5	Equipo C6 - Equipo B3	C-B	Sábado 12:00	0.75	0	0.55	0.4125
6	Equipo B5 - Equipo B6	B-B	Sábado 16:00	0.90	0	0.70	0.6300
7	Equipo B7 - Equipo B11	B-B	Sábado 18:00	0.90	0	0.80	0.7200
8	Equipo A1 - Equipo B1	A-B	Sábado 20:00	1.30	0	1.00	1.3000
9	Equipo C2 - Equipo C5	C-C	Viernes 20:00	0.47	0	0.40	0.1880

10. Referencias

Enumera las referencias utilizadas

- Universidad Internacional de Valencia. *Trabajo Práctico - SE1: Algoritmos de Optimización*. Enunciado y plantilla proporcionados en el aula.
- Python Software Foundation. *itertools — Functions creating iterators*. https://docs.python.org/3/library/itertools.html
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.
- Hardy, G. H., Littlewood, J. E. y Pólya, G. (1952). *Inequalities* (2.ª ed.). Cambridge University Press.

Declaración de uso de inteligencia artificial

Utilicé OpenAI Codex como apoyo para revisar el enunciado, contrastar el modelo matemático, depurar el código y revisar la redacción. La selección del enfoque, la ejecución de las pruebas y la validación final del trabajo las realicé personalmente.

11. Líneas de trabajo futuras

Describe brevemente cómo es posible avanzar en el estudio del problema

El modelo podría ampliarse sustituyendo las categorías fijas por estimaciones de audiencia construidas a partir de datos históricos. Entre las variables relevantes podrían incluirse la audiencia previa de cada equipo, el histórico de enfrentamientos, la rivalidad, la clasificación actual, la forma reciente, la importancia deportiva del partido y la franja horaria.

También sería interesante trabajar en la estimación de la pérdida de audiencia provocada por las coincidencias. En el modelo actual, la penalización depende únicamente del número de partidos simultáneos. Sin embargo, el efecto real puede variar según los equipos implicados y el interés que generen los encuentros que compiten en una misma franja.

Con estas mejoras, el modelo podría recalibrarse conforme avanzara la temporada. La comparación entre las audiencias estimadas y las realmente obtenidas permitiría medir el error, ajustar los parámetros y comprobar si la programación propuesta mejora los resultados frente a la asignación utilizada habitualmente.